

model-profiler Developer Documentation

This document is for contributors and maintainers who need to understand how `model-profiler` works internally, how memory is calculated, and how features are organized.

1) Project Purpose and Scope

`model-profiler` estimates **inference memory requirements** from model weight metadata without loading full tensors into memory.

Primary goals:

- Keep the core dependency footprint minimal.
- Support multiple model sources with a single execution flow.
- Estimate model memory from `safetensors` metadata and PyTorch checkpoint metadata.
- Optionally estimate KV cache memory for decoder-style models.

Out of scope:

- Runtime profiling of live inference servers.
 - Activation-memory estimation during forward/backward passes.
 - Exact engine-specific allocator behavior (the output is an estimate, not a simulator).
-

2) Repository Structure

- `src/model_profiler/cli.py`
 - Entry point (`model-profiler` command), argument parsing, connector selection, and orchestration.
 - `src/model_profiler/connectors/`
 - Source-specific file access adapters: HF Hub, local filesystem, S3, GCS, Azure.
 - `src/model_profiler/metadata.py`
 - Safetensors header parsing and aggregation into per-component/per-dtype totals.
 - `src/model_profiler/pytorch_bin.py`
 - Metadata-only parsing of `pytorch_model.bin` (streaming or full read).
 - `src/model_profiler/types.py`
 - Dtype mappings and byte-size definitions.
 - `src/model_profiler/print.py`
 - Human-readable report rendering (ASCII table with bars).
 - `README.md`
 - User-facing usage documentation.
 - `pyproject.toml`
 - Package metadata, dependencies, extras, script entry point.
-

3) End-to-End Execution Flow

The CLI follows a single high-level path:

1. Parse CLI arguments.
2. Resolve connector (explicit `--connector` or inferred from source args).
3. Build selected connector instance.
4. List files from source via connector.
5. Detect supported model layout.
6. Read minimal metadata for weights.
7. Convert metadata into totals (params and bytes).
8. Optionally compute quantization label and KV cache estimate.
9. Output either:
 - Pretty table (`print_report`), or
 - JSON payload (`--json-output`).

Core orchestration functions:

- `main()` parses args and dispatches.
- `run()` handles default HF connector setup.
- `run_with_connector()` does format detection + metadata + optional extras.

4) Supported Model Layouts and Detection Order

`run_with_connector()` checks files in this order:

1. `model.safetensors`
2. `model.safetensors.index.json` (sharded safetensors)
3. `model_index.json` (Diffusers-style components)
4. `pytorch_model.bin`
5. `pytorch_model.bin.index.json` (sharded PyTorch bins)

If none are present, it raises a `RuntimeError`.

Why this matters:

- The order determines which parser path is taken when repositories contain multiple formats.
- Each path normalizes to a shared metadata model before reporting.

5) Memory Estimation Model

5.1 Weight Memory

Weight memory is derived from metadata only:

- For each tensor:
 - `param_count = product(shape)`
 - `bytes_count = param_count * bytes_per_dtype`

Totals are accumulated:

- Per dtype (e.g., F16/BF16/F32).
- Per component (Transformer, Diffusers submodule, SentenceTransformer dense blocks).
- Global totals across components.

No tensor payload bytes are loaded for this computation.

5.2 KV Cache Memory (`--experimental`)

When `--experimental` is enabled and a `config.json` exists, KV cache estimate is computed as:

```
cache_size = 2 * num_hidden_layers * num_kv_heads * head_dim * max_model_len *
dtype_bytes * batch_size
```

Where:

- `2` accounts for keys + values.
- `num_kv_heads` falls back to `num_attention_heads` if missing.
- `head_dim` falls back to `hidden_size // num_attention_heads` if not explicitly present.
- `max_model_len` is inferred from config when possible, otherwise defaulted to `131072` with warning.
- `dtype_bytes` defaults to `2` and is user-configurable.

Combined estimate shown to user:

- `total_memory = weight_bytes + cache_size` (when cache is enabled/available).

Important note:

- This cache estimate is architecture-parameter-based and does not model paged attention, block manager overhead, or fragmentation.

5.3 Quantization Label Resolution

`resolve_quantization_config()` translates known quantization config patterns into a compact label:

- `awq, gptq` -> `int{bits}`
- `bitsandbytes` -> `int8` or `int4`
- `fp8, mxfp4, nvfp4`
- TensorRT Model Optimizer style config

If missing/unknown, behavior defaults to `float16` or `None` depending on path.

6) How Memory Is Managed Internally

This section explains the tool's **own runtime memory behavior** (not just model estimates).

6.1 Safetensors Header-Only Reads

For each safetensors file, `model-profiler` reads only header metadata:

- Initial read uses `MAX_METADATA_SIZE` bytes (default `100000`).
- If metadata is larger, a second read fetches only the remaining bytes needed.

Result:

- Avoids downloading entire `.safetensors` files.
- Keeps peak process memory low for large models.

6.2 Range Reads Through Connectors

All connectors support `read_file(path, offset, limit)` so parsers can request only needed byte ranges.

This enables:

- Partial header reads for `safetensors`.
- Streaming metadata extraction for `.bin` zip structures.

6.3 Streaming PyTorch `.bin` Metadata

`pytorch_bin.py` supports a no-torch, metadata-only path:

1. Read file tail to locate ZIP EOCD.
2. Read central directory.
3. Locate `data.pkl/data`.
4. Read only pickle payload.
5. Unpickle using stub classes that preserve shape and dtype metadata.

Only if this path fails does the CLI fall back to:

- Full file read + torch-based load (`map_location="meta", weights_only=True`) if torch extra is installed.

This design minimizes full-file allocations whenever possible.

6.4 Concurrency and Network Behavior

- Async HTTP client is configured with connection pooling and HTTP/2.
- `MAX_CONCURRENCY` defaults to `min(32, os.cpu_count()+4)` and can be overridden via `MAX_WORKERS`.
- Timeout controlled by `REQUEST_TIMEOUT` env var.

Concurrency is used for:

- Parallel metadata fetches in sharded `safetensors` paths.

Trade-off:

- Higher concurrency can reduce wall-clock latency but increase short-lived memory/network pressure.

6.5 CPU vs Event Loop Boundaries

Blocking operations are moved off the event loop with `asyncio.to_thread`:

- Local filesystem access.
- Cloud SDK calls.
- CPU-side parsing fallback paths.

This keeps async orchestration responsive while preserving a simple codebase.

7) Connector Architecture

All connectors implement the shared protocol in `connectors/base.py`:

- `list_files()`
- `read_file(path, offset=0, limit=None)`
- `read_file_json(path)`
- `get_file_size(path)`

Available connectors:

- `HFCConnector`
 - Uses Hugging Face API tree endpoint + resolve URLs.
 - Optional auth from `HF_TOKEN` or token file in `HF_HOME`.
- `LocalConnector`
 - Walks local directory recursively.
- `S3Connector` (optional extra: `boto3`)
 - Supports prefix scoping and range reads.
- `GCSConnector` (optional extra: `google-cloud-storage`)
 - Supports prefix scoping and range reads.
- `AzureConnector` (optional extra: `azure-storage-blob`, `azure-identity`)
 - Auth via connection string or account + `DefaultAzureCredential`.

Design benefit:

- Parsing logic is source-agnostic and reused for all storage backends.
-

8) Feature Inventory

8.1 Core Features

- Inference memory estimate from safetensors metadata.
- Sharded safetensors support.
- Diffusers multi-component model support via `model_index.json`.
- Sentence Transformers Dense module handling via `modules.json`.
- JSON output mode (`--json-output`).
- Human-readable table output.
- Tensor-parallel constraints extraction (`--tp-limits`).

8.2 PyTorch Checkpoint Features

- `pytorch_model.bin` metadata extraction without torch dependency (best path).
- Streaming ZIP central-directory approach to avoid full reads.
- Fallback to torch metadata load for incompatible files (`model-profiler[pytorch]`).
- Sharded `.bin` via `pytorch_model.bin.index.json`.

8.3 Experimental/Advanced Features

- KV cache estimate (`--experimental`).
- Adjustable `--max-model-len`, `--batch-size`, `--dtype-bytes`.
- Quantization label extraction from `config.json` / `hf_quant_config.json`.

8.4 Source/Connector Features

- HF Hub support (default).
- Local path support.
- S3, GCS, Azure support via optional extras.
- Connector inference from source flags.

8.5 Operational Features

- Verbose mode (`-v/--verbose`) for loader path diagnostics.
- Configurable request timeout and worker concurrency by environment variables.
- Graceful handling of missing optional dependencies with actionable error messages.

9) CLI Arguments (Developer View)

Main source selection:

- `--connector hf|local|s3|gcs|azure`
- `--model-id`
- `--revision`
- `--local-path`
- `--s3-bucket`, `--s3-prefix`
- `--gcs-bucket`, `--gcs-prefix`
- `--azure-container`, `--azure-prefix`, `--azure-account`

Memory/report controls:

- `--experimental`
- `--max-model-len`
- `--batch-size`
- `--dtype-bytes`
- `--json-output`
- `--ignore-table-width`
- `--tp-limits`
- `--verbose`

Implementation detail:

- For HF default path, `run()` creates a tuned `httpx.AsyncClient`.
 - For non-HF paths, `run_with_connector()` is used directly.
-

10) Error Handling and Fallback Strategy

Typical failure categories:

- Unsupported repository layout (no recognized model files).
- Missing cloud SDK extras for selected connector.
- Auth/network failures for remote sources.
- Unsupported/legacy `.bin` formats without torch fallback installed.

Fallback patterns:

- Prefer no-torch + streaming for `.bin`; then fallback to full read + torch path.
- Warn (not fail) when optional config keys for experimental estimate are missing.

Developer guidance:

- Preserve current fallback order when changing loader paths.
 - Keep errors actionable (e.g., exact `pip install model-profiler[...]` hint).
-

11) Dependency Philosophy

From `pyproject.toml`:

- Core dependency: `httpx[http2]`.
- Optional extras for source connectors and torch fallback parser.

Intent:

- Keep base install lightweight.
 - Allow opt-in integration for cloud/object-storage and legacy `.bin` support.
-

12) Performance Characteristics

Complexity drivers:

- Number of model files/shards.
- Header size for safetensors.
- Network round trips and source latency.

Performance-oriented choices already in code:

- Range reads for metadata.
- Async concurrent shard metadata fetch for safetensors.
- Streaming `.bin` metadata extraction path.

Potential future optimization areas:

- Configurable limits for shard concurrency at gather-call granularity.
 - Retry/backoff policy customization for transient remote failures.
-

13) Extending the Project

13.1 Add a New Connector

1. Implement connector protocol methods.
2. Add connector import/export in `connectors/__init__.py`.
3. Add CLI options for source-specific args.
4. Extend connector resolution logic in `main()`.
5. Update README and this doc.

13.2 Add New Quantization Formats

1. Extend `resolve_quantization_config()`.
2. Keep outputs in compact, user-friendly labels.
3. Ensure unknown formats degrade gracefully.

13.3 Add New Model Layout Support

1. Detect new marker files in `run_with_connector()`.
 2. Convert to normalized `raw_metadata` shape expected by `parse_safetensors_metadata()`.
 3. Maintain existing precedence and avoid regressions.
-

14) Testing and Validation Recommendations

This repository currently emphasizes lean implementation over heavy test scaffolding. For safe changes:

- Validate against each connector path (HF/local at minimum).
 - Validate each model format path:
 - single safetensors,
 - sharded safetensors,
 - diffusers index,
 - `.bin`,
 - sharded `.bin`.
 - Validate both output modes:
 - table,
 - JSON.
 - Validate `--experimental` with and without complete config keys.
 - Validate `--tp-limits` on decoder-only and conditional generation configs.
-

15) Known Limitations

- Estimates are static and metadata-based; runtime allocator overhead and fragmentation are not modeled.
 - KV cache logic is intentionally approximate.
 - Some legacy `.bin` variations may require torch fallback.
 - Report formatting is optimized for terminal readability, not machine parsing (use JSON mode for automation).
-

16) Quick Reference for Maintainers

- Core orchestrator: `src/model_profiler/cli.py`
- Metadata parser: `src/model_profiler/metadata.py`
- `.bin` parser: `src/model_profiler/pytorch_bin.py`
- Connectors: `src/model_profiler/connectors/*`
- Output renderer: `src/model_profiler/print.py`
- Package/deps: `pyproject.toml`

When debugging memory-estimation differences:

1. Confirm which model layout branch was selected.
 2. Check dtype mapping output and byte multipliers.
 3. Check whether cache estimate is included.
 4. For `.bin`, check whether streaming path or torch fallback path was used (`--verbose`).
-

17) Integration with Dobby Profile Generation

This section documents how `/home/ubuntu/dobby/dobby-utils/dobby_utils/profiles.py` uses `model-profiler` data to generate deployable profiles and "best fit" combinations.

17.1 Entry Point and Contract

In `dobby_utils/profiles.py`, profile generation starts with:

- `generate_profile(ProfileRequest)`

The request carries:

- `source: ModelSource` (HF/S3/GCS source with credentials when required)

The profiler output consumed from `model-profiler` is obtained through:

- `get_model_size_info(source)` in `dobby_utils/utils.py`
- which internally calls `model_profiler.cli.run_with_connector(..., json_output=True, tp_limits=True, experimental=True)`

So Dobby consumes:

- `param_count`
- `bytes_count`
- `components` and dtype map

- `cache_size` (experimental KV cache estimate)
- `architecture`
- `tp_constraints.recommended_for_single_node`
- optional `quantization`

17.2 Profile Generation Pipeline

`generate_profile()` follows this flow:

1. Resolve model memory and architecture via `get_model_size_info`.
2. Extract TP candidates from `tp_constraints.recommended_for_single_node`.
3. Resolve default quantization:
 - from `model_info["quantization"]` if available, else
 - inferred from model dtypes using `get_dtype_mapping(...)`.
4. Map architecture to Dobby model type via `HF_ARCH_TO_Dobby_TYPE`.
5. Resolve allowed machine/quantization combinations via `QMA_MAPPING`.
6. For each machine + TP candidate (+ quantization candidate), check memory fit.
7. Emit valid profile entries and aggregate them into:
 - `quantizations` map (quantization -> supported machines)
 - `combinations` list (quantization + machine + allowed parallelism lists)

If no valid entry survives constraints, `U015` is raised (no compatible profile).

17.3 Memory Fit Formula Used for Candidate Filtering

For each candidate:

- Weight footprint in GB:
 - `bytes_count_per_gb = bytes_count / 1024^3`
 - with scaling when trying lower-bit quantizations (`int4`, `fp8`) in float16 fallback branch.
- KV cache footprint in GB:
 - `(dtype_factor * kv_cache_size) / 1024^3`
- Total estimated footprint must satisfy:
 - `estimated_weight_gb + estimated_kv_gb <= tensor_parallelism * machine_vram_gb`

Machine VRAM and max TP limits come from `MACHINE_MAPPING`.

17.4 How "Best Profiles" Are Produced

Current code generates **all valid profiles**, then groups them into normalized combinations. It does not do an explicit single "best score" ranking in `profiles.py` yet.

Practical interpretation for "best profile":

- Lowest quantization loss that still fits target machine VRAM.
- Smallest TP degree that fits (simpler deployment and lower communication overhead).
- Preferred machine family for your infra/cost constraints.

Recommended deterministic ranking (for future addition):

1. Prefer higher numerical precision (`float16` > `fp8` > `int8` > `int4`) when all fit.
2. Prefer lower TP (1, then 2, then 4, ...).
3. Prefer lower GPU count for equivalent fit.
4. Tie-break by machine priority configured per organization.

17.5 Why This Works Well with `model-profiler`

This integration is reliable because `model-profiler` provides:

- source-agnostic memory extraction (HF/S3/GCS already supported),
- architecture and TP constraints,
- experimental KV cache estimate for sequence-heavy models,
- quantization hints from model config when available.

That gives Dobby enough signal to produce deployment-ready candidate profiles without loading full model weights.

17.6 Extension Points

If you want better automatic "best profile" selection, add:

- a ranking function after `profiles` list creation in `generate_profile()`,
- policy weights for cost/performance/latency,
- architecture-specific penalties (for unsupported or unstable quantizations),
- optional runtime benchmark feedback loop to rerank static memory-based candidates.